

React-Table Sort with Pagination Unexpected Results

Asked 3 years, 6 months ago Modified 1 year, 8 months ago Viewed 4k times



2



Getting some unexpected results while using react-table. I've combined examples for pagination-controlled and sorting from [react-table examples](#). The issue i'm experiencing is the table hooks seem to be resetting the page index back to default on a second re-render. This is resulting in fetchData being called twice when paging back and forth. Below is a reproduction of the problem i'm facing:



```
import React from "react";
import styled from "styled-components";
import { useTable, usePagination, useSortBy } from "react-table";

import makeData from "../makeData";

const Styles = styled.div`

// Let's add a fetchData method to our Table component that will be used to fetch
// new data when pagination state changes
// We can also add a loading state to let our table know it's loading new data
function Table({
  columns,
  data,
  fetchData,
  loading,
  pageCount: controlledPageCount
}) {
  const {
    getTableProps,
    getTableBodyProps,
    headerGroups,
    prepareRow,
    page,
    canPreviousPage,
    canNextPage,
    pageOptions,
    pageCount,
    gotoPage,
    nextPage,
    previousPage,
    setPageSize,
    // Get the state from the instance
    state: { pageIndex, pageSize }
  } = useTable(
    {
      columns,
      data,
      initialState: { pageIndex: 0 }, // Pass our hoisted table state
      manualPagination: true, // Tell the usePagination
      // hook that we'll handle our own data fetching
      // This means we'll also have to provide our own
      // pageCount.
      pageCount: controlledPageCount
    },
    useSortBy,
    usePagination
  );
```

```

// Listen for changes in pagination and use the state to fetch our new data
React.useEffect(() => {
  fetchData({ pageIndex, pageSize });
}, [fetchData, pageIndex, pageSize]);

// Render the UI for your table
return (
  <>
    <pre>
      <code>
        {JSON.stringify(
          {
            pageIndex,
            pageSize,
            pageCount,
            canNextPage,
            canPreviousPage
          },
          null,
          2
        )}
      </code>
    </pre>
    <table {...getTableProps()}>
      <thead>
        {headerGroups.map(headerGroup => (
          <tr {...headerGroup.getHeaderGroupProps()}>
            {headerGroup.headers.map(column => (
              <th {...column.getHeaderProps(column.getSortByToggleProps())}>
                {column.render("Header")}
                <span>
                  {column.isSorted
                    ? column.isSortedDesc
                      ? " ▾"
                      : " ▴"
                    : ""}
                </span>
              </th>
            ))}
          </tr>
        ))}
      </thead>
      <tbody {...getTableBodyProps()}>
        {page.map((row, i) => {
          prepareRow(row);
          return (
            <tr {...row.getRowProps()}>
              {row.cells.map(cell => {
                return (
                  <td {...cell.getCellProps()}>{cell.render("Cell")}</td>
                );
              })}
            </tr>
          );
        })}
      <tr>
        {loading ? (
          // Use our custom loading state to show a loading indicator
          <td colSpan="10000">Loading...</td>
        ) : (
          <td colSpan="10000">
            Showing {page.length} of ~{controlledPageCount * pageSize}{" "}
            results
          </td>
        )}
      </tr>
    </tbody>
  </table>
)

```

```

    </tbody>
  </table>
  /*
    Pagination can be built however you'd like.
    This is just a very basic UI implementation:
  */
  <div className="pagination">
    <button onClick={() => gotoPage(0)} disabled={!canPreviousPage}>
      {"<<"}
    </button>{" "}
    <button onClick={() => previousPage()} disabled={!canPreviousPage}>
      {"<"}
    </button>{" "}
    <button onClick={() => nextPage()} disabled={!canNextPage}>
      {">"}
    </button>{" "}
    <button onClick={() => gotoPage(pageCount - 1)} disabled={!canNextPage}>
      {">>"}
    </button>{" "}
    <span>
      Page{" "}
      <strong>
        {pageIndex + 1} of {pageOptions.length}
      </strong>{" "}
    </span>
    <span>
      | Go to page:{" "}
      <input
        type="number"
        defaultValue={pageIndex + 1}
        onChange={e => {
          const page = e.target.value ? Number(e.target.value) - 1 : 0;
          gotoPage(page);
        }}
        style={{ width: "100px" }}
      />
    </span>{" "}
    <select
      value={pageSize}
      onChange={e => {
        setPageSize(Number(e.target.value));
      }}
    >
      {[10, 20, 30, 40, 50].map(pageSize => (
        <option key={pageSize} value={pageSize}>
          Show {pageSize}
        </option>
      ))}
    </select>
  </div>
</>
);
}

// Let's simulate a large dataset on the server (outside of our component)
const serverData = makeData(10000);

function App() {
  const columns = React.useMemo(
    () => [
      {
        Header: "Name",
        columns: [
          {
            Header: "First Name",
            accessor: "firstName"

```

```

    },
    {
      Header: "Last Name",
      accessor: "lastName"
    }
  ]
},
{
  Header: "Info",
  columns: [
    {
      Header: "Age",
      accessor: "age"
    },
    {
      Header: "Visits",
      accessor: "visits"
    },
    {
      Header: "Status",
      accessor: "status"
    },
    {
      Header: "Profile Progress",
      accessor: "progress"
    }
  ]
}
],
[]
);

// We'll start our table without any data
const [data, setData] = React.useState([]);
const [loading, setLoading] = React.useState(false);
const [pageCount, setPageCount] = React.useState(0);
const fetchIdRef = React.useRef(0);

const fetchData = React.useCallback(({ pageSize, pageIndex }) => {
  // This will get called when the table needs new data
  // You could fetch your data from literally anywhere,
  // even a server. But for this example, we'll just fake it.

  // Give this fetch an ID
  const fetchId = ++fetchIdRef.current;

  // Set the loading state
  setLoading(true);

  // We'll even set a delay to simulate a server here
  setTimeout(() => {
    // Only update the data if this is the latest fetch
    if (fetchId === fetchIdRef.current) {
      const startRow = pageSize * pageIndex;
      const endRow = startRow + pageSize;
      setData(serverData.slice(startRow, endRow));

      // Your server could send back total page count.
      // For now we'll just fake it, too
      setPageCount(Math.ceil(serverData.length / pageSize));

      setLoading(false);
    }
  }, 1000);
}, []);

```

```
return (  
  <Styles>  
    <Table  
      columns={columns}  
      data={data}  
      fetchData={fetchData}  
      loading={loading}  
      pageCount={pageCount}  
    />  
  </Styles>  
);  
}  
  
export default App;
```

Here is a codesandbox to observe the behavior: [codesandbox](#)

Also worth noting that the issue does not occur after removing the useSort hook along with call to column.getSortByToggleProps().

javascript

reactjs

react-table

Share Improve this question Follow

asked May 7, 2020 at 1:34



Mike Lunn

Mike

2,350 ● 3 ● 21 ● 25

Came across the same problem, here is a detailed discussion on this topic:

github.com/tannerlinsley/react-table/issues/411 – Nirojan Selvanathan Aug 15, 2021 at 14:53

1 Answer

Sorted by: Highest score (default) ▾



By mutating `data` with every `fetchPage` request, you signal to something in react-table that "hey, our data refreshed! It must be new data! Reset the page!"

11



[The usePagination docs](#) mention an `autoResetPage` property -- setting it `false` enables the expected behavior. Note that it also disables page-resetting on sort, filter, and grouping, so additional logic may be required if you plan on using those as well.



Share Improve this answer

edited Feb 14, 2022 at 0:05

answered Jun 5, 2020 at 3:03

Follow



toki

946 ● 5 ● 18

Thanks mate you save my time – Gautam Patadiya Feb 9, 2021 at 10:35

- 1 This is the correct answer. You can also read here for more info [react-table.tanstack.com/docs/...](https://react-table.tanstack.com/docs/) – Carlos Henriquez Feb 11, 2022 at 20:03